

Handout 1

Verified Compiler Correctness

Software Verification

Nova School of Science and Technology
Mário Pereira `mjp.pereira@fct.unl.pt`

Version of September 27, 2026

The goal of this handout is to verify a Rocq implementation of a toy compiler from a language of arithmetic expressions into a stack machine. This assignment can be roughly divided into three main bulks of work: first, equipping both languages, *i.e.*, the *source* arithmetic language and the *target* machine language, with proper *small-step* semantics; second, defining a compilation function from the source to the target language; finally, proving *semantic preservation* of the said compilation function.

Semantic preservation is the main result when proving the correctness of compilers. It states that, if one fully executes an expression e of the source language and it produces a value v , then it must be the case that if one executes the machine program resulting from compiling e it must also produce v . In our case, for our small source arithmetic language, values are limited to integer constants. However, even if in a very different scale, this is exactly the approach the proof of a realistic compiler takes. The main examples are COMPCERT¹, for the C language, and CAKEML², for an ML-like language.

During this handout you are supposed to complete definitions and proofs, given in the companion `.v` file.

Each exercise in this handout poses a challenge to be addressed in the given Rocq file: either a definition that you must complete, or a proof that you must finish. In the `.v` file, you will find several times the comment

`(* FILL HERE *)`

with the corresponding exercise number. This is exactly where you are supposed to write your solution.

¹<https://compcert.org>

²<https://cakeml.org>

1 Source Language

1.1 Syntax

Our source language is a very simple arithmetic language. It features only integer literals, addition, and subtraction. The *concrete syntax* is as follows:

$$e ::= n \mid e + e \mid e - e$$

In order to represent terms of such language in Rocq, we introduce the following *abstract syntax* in the form of an AST (Abstract Syntax Tree):

```
Inductive op : Type :=                                     Rocq
| OAdd
| OSub.

Inductive expr : Type :=
| EConst: Z → expr
| EBinOp : expr → op → expr → expr.
```

Note that we have factorized addition and subtraction into a single constructor EBinOp, which stands for binary operations. In Rocq, type Z stands for the type of integer values.

1.2 Operational Semantics

1.2.1 Single-step Reduction

In order to mathematically describe the behavior of a language, we must devise a *formal semantics* for such a language. In our case, we use a *small-step* style operational semantics, which expresses a single step of execution. In other words, an expression e *reduces* to another expression e' in a *single step* of execution. This is exactly the behavior of a debugger.

The semantics of expressions is described as a predicate $\text{red_expr } e \ e'$, defined via the following inference rules:

$$\frac{n = \text{eval_op } n_1 \circ n_2}{\text{red_expr } (n_1 \circ n_2) \ n} \text{ (RED_BETA)}$$
$$\frac{\text{red_expr } e_1 \ e'_1}{\text{red_expr } (e_1 \circ e_2) \ (e'_1 \circ e_2)} \text{ (RED_LEFT)}$$
$$\frac{\text{red_expr } e_2 \ e'_2}{\text{red_expr } (n_1 \circ e_2) \ (n_1 \circ e'_2)} \text{ (RED_RIGHT)}$$

Here, the placeholder $\circ$ stands for either addition, $+$, or subtraction, $-$. Rule (RED_BETA) states that when we have integer constants n_1 and n_2

on both sides of the operation we compute a new integer n . Depending on the case of $\circ$, this can be either $n_1 + n_2$ or $n_1 - n_2$. We use the auxiliary function `eval_op` to exactly compute such a value.

Rule (RED_LEFT) stands for a reduction when the first operand is not yet an integer constant. Under this case, we first take a computation step to reduce e_1 to a new expression e'_1 . Finally, rule (RED_RIGHT) stands for a reduction when the first operand is already a constant, but the second one is not. In such a case, we take a step of computation to reduce e_2 into e'_2 .

Exercise 1. Complete the following Rocq definition

Inductive `red_expr` : `expr` $\rightarrow$ `expr` $\rightarrow$ `Prop` := . *Rocq*

according to rules (RED_BETA), (RED_LEFT), and (RED_RIGHT).

Note that the rules above use the concrete syntax of our arithmetic language. In Rocq, you must use the abstract syntax constructors to express the language's operational semantics. $\square$

1.2.2 Many-steps Evaluation

Small-step operational semantics is very good at describing the individual steps of computation, but not the best to capture the final result of fully reducing an expression into a value. In other words, what is the result of *many steps of reduction* over an expression.

To relate an expression e with the integer value n it produces when fully evaluated, we shall introduce the relation `red_expr_star e n`. This predicate reads as “expression e reduces to the value n after zero or more steps of execution”. Such a predicate is defined via the following inference rules:

$$\frac{\text{(RED_STAR_ZERO)}}{\text{red_expr_star } n \ n} \quad \frac{\text{(RED_STAR_STEP)} \quad \text{red_expr } e \ e' \quad \text{red_expr_star } e' \ n}{\text{red_expr_star } e \ n}$$

Rule (RED_STAR_ZERO) reads as follows: a integer constant n is already fully reduced; it reduces to itself in zero steps. Rule (RED_STAR_STEP) reads as follows: if an expression e reduces, in a step, to an expression e' and expression e' reduces to an integer value n in many steps, then expression e reduces to n in many steps.

For the attentive reader, the relation induced by predicate `red_expr_star` is the *transitive-reflexive* closure of `red_expr`.

Exercise 2. Complete the following Rocq definition

Inductive `red_expr_star` : `expr` $\rightarrow$ $\mathbb{Z}$ $\rightarrow$ `Prop` := . *Rocq*

according to rules (RED_STAR_ZERO) and (RED_STAR_STEP). $\square$

2 Target Stack-Base Machine Language

2.1 Syntax

The target language of our toy compiler is a stack-based machine. Such a machine works by pushing and popping values from a stack, following a list of instructions as in an Assembly language. Our target language instruction set features only three constructions, as expressed by its abstract syntax:

```
Inductive asm : Type :=                                     Rocq
| APush : Z → asm
| AAdd : asm
| ASub : asm.
```

The first constructor, `APush`, takes as argument a single integer value. The other two constructors, `AAdd` and `ASub`, are used to instruct the machine to add and subtract, respectively, the top elements of a stack of integer values.

Consider the following, valid program written for our machine, expressed using lists notation: `[APush 22; APush 22; AAdd; APush 2; ASub]`. Let us explain, informally, how it evaluates. The first `APush` instruction puts the integer 22 on top of the *stack*. The stack here is literary a LIFO data structure, where one pushes values on top and pops also from the top. This stack is part of the execution environment, akin to some form of RAM memory in a CPU. The second `APush` also puts 22 on the stack. At this point, the stack is a list of the form `[22; 22]`. Afterwards, the `AAdd` instruction pops the two top values of the stack and puts back the sum, hence the stack is now `[44]`. Finally, the last `APush` puts 2 on top and `ASub` subtracts 44 to 2, putting back 42 on top of the stack. The result of executing this program is the final stack `[42]`.

2.2 Operational Semantics

2.2.1 Single-step Reduction

We shall define the operational semantics of our machine language, again, as a small-step relation. First, we introduce the type of *outcomes* of evaluation:

```
Definition stack : Type := list Z.                             Rocq
```

```
Inductive outcome : Type :=
| ORes : stack → outcome
| OAbort : outcome.
```

The first case represents a *well-succeeded* evaluation, one that results in a final stack. The second case, `OAbort`, stands for a machine evaluation error, namely when the stack does not contain enough elements to proceed with addition or subtraction.

The single-step reduction for our machine language is defined via the following function:

$$\begin{aligned}
\text{red_asm } s \text{ (APush } n) &= \text{ORes } (n :: s) \\
\text{red_asm } (n_1 :: n_2 :: s) \text{ AAdd} &= \text{ORes } ((n_2 + n_1) :: s) \\
\text{red_asm } (n_1 :: n_2 :: s) \text{ ASub} &= \text{ORes } ((n_2 - n_1) :: s) \\
\text{red_asm } s \text{ a} &= \text{OAbort}
\end{aligned}$$

The last rule stands for an *otherwise* case, when the stack s does not contain at least two elements.

Exercise 3. Complete the following Rocq definition

Definition `red_asm (s: stack) (a: asm) : outcome := OAbort.` *Rocq*

according to the `red_asm` function defined above. If, in the body of your function, you try to define an arithmetic operation of the form `n1 + n2` and Rocq refuses saying it is expecting a value of type `natural`, try writing `(n1 + n2)%Z`. □

2.2.2 Many-steps Evaluation

We shall now define the *many-steps* evaluation for our stack machine. This is done via the following *recursive function*:

$$\begin{aligned}
\text{red_asm_star } s \text{ []} &= \text{ORes } s \\
\text{red_asm_star } s \text{ (a :: c')} &= \begin{cases} \text{OAbort}, & \text{if } \text{red_asm } s \text{ a} = \text{OAbort} \\ \text{red_asm_star } s' \text{ c'}, & \text{if } \text{red_asm } s \text{ a} = \text{ORes } s' \end{cases}
\end{aligned}$$

The `red_asm_star` function takes as arguments a stack s and a list of machine instructions c . If c is the empty list, then the evaluation is done, the outcome being `ORes s`, *i.e.*, a successful computation where the final result is the stack s . If c is of the form `a :: c'`, then we proceed by calling function `red_asm s a`. If it returns `OAbort`, then we must propagate this outcome as the result of the initial call `red_asm_star s c`. Otherwise, we continue recursively when `red_asm s a` returns `ORes s'`.

Exercise 4. Complete the following Rocq definition

Definition `code : Type := list asm.` *Rocq*

Fixpoint `red_asm_star (s: stack) (c: code) : outcome := OAbort.`

according to the `red_asm_star` function defined above. □

3 Code Compilation

We arrive at the core of our toy compiler: how to compile an arithmetic expression down to a program written for the stack-based machine. Such a translation schema is defined as follows:

$$\begin{aligned}\text{compile } n &= [\text{APush } n] \\ \text{compile } (e_1 \circ e_2) &= \text{compile } e_1 ++ \text{compile } e_2 ++ [\text{compile_op } \circ]\end{aligned}$$

where $++$ is the standard list concatenation and compile_op is simply defined as follows:

$$\begin{aligned}\text{compile_op } (+) &= \text{AAdd} \\ \text{compile_op } (-) &= \text{ASub}\end{aligned}$$

One can think of compiling into a stack machine as a process of “compiling first the operands of every intermediate operation, and then execute the actual arithmetic operation”. Let us use the expression $22 + 22 - 2$ to illustrate how this compilation schema works. Following the above definition, $\text{compile } (22 + 22 - 2)$ would produce the following program:

$$[\text{APush } 22; \text{APush } 22; \text{AAdd}; \text{APush } 2; \text{ASub}]$$

Executing this program is exactly as seen in Section 2.1.

Exercise 5. Complete the following Rocq definition

Definition $\text{compile_op } (o: \text{op}) : \text{asm} :=$ *Rocq*
 $\text{match } o \text{ with}$
 $\quad | \text{OAdd} \Rightarrow \text{AAdd}$
 $\quad | \text{OSub} \Rightarrow \text{ASub}$
 end.

Fixpoint $\text{compile } (e: \text{expr}) : \text{code} := [].$

according to the compile function defined above. □

4 Properties About Expressions Semantics

In this section, you are asked to prove a simple property about the many-steps evaluation of arithmetic expressions.

Exercise 6. Prove the following Lemma about the red_expr_star relation:

Lemma $\text{red_expr_star_const_inv} : \forall n m: \mathbb{Z},$ *Rocq*
 $\text{red_expr_star } (\text{EConst } n) m \rightarrow n = m.$

□

In the companion Rocq file you are provided the following Lemma:

Parameter `red_expr_star_middle` : $\forall e1\ e2\ n\ op,$ *Rocq*
`red_expr_star (EBinOp e1 op e2) n  $\rightarrow$`
 `$\exists n1\ n2,$ red_expr_star e1 n1 $\wedge$ red_expr_star e2 n2 $\wedge$
 $n = eval_op\ n1\ op\ n2.$`

The following result can be read as follows: if $e_1 \circ e_2$ evaluates to n in many steps, then there exist integer values n_1 and n_2 such that e_1 evaluates to n_1 , e_2 evaluates to n_2 , and $n = eval_op\ n_1\ \circ\ n_2$.

You do not need to prove this result, but you are supposed to use it in later proofs. A **Parameter** definition in Rocq works like an axiom: some result that you can use without proving it.

5 Properties About Stack Machine Evaluation

In this section, you are asked to prove some properties about the operational semantics of our stack-based machine.

Exercise 7. Prove the following Lemma about the compilation of binary operators:

Lemma `red_asm_compile_op` : $\forall (o: op)\ (n1\ n2: Z)\ (s: stack),$ *Rocq*
`red_asm (n2 :: n1 :: s) (compile_op o) = ORes (eval_op n1 o n2 :: s).`

Proceed by case analysis on the operator o . $\square$

Exercise 8. Prove the following Lemma about the many-steps evaluation of programs in the stack machine:

Lemma `comm_red_asm_append` : $\forall (c1\ c2: code)\ (s0\ s1: stack),$ *Rocq*
`red_asm_star s0 c1 = ORes s1  $\rightarrow$`
`red_asm_star s0 (c1 ++ c2) = red_asm_star s1 c2.`

Proceed by structural induction on the code $c1$.

This result is of particular importance. During execution in the stack machine, it is often useful to “break in the middle”. This Lemma allows you exactly to do that. $\square$

6 Compiler Correctness

We finally arrive at the main point of our Rocq development: proving that our compiler from arithmetic expressions into a stack machine is correct. Very briefly, proving that a compiler is correct boils down to show that a compiler respects the semantics. If the input language is equipped with a semantics $\rightarrow_s$ and the target language with a semantics $\rightarrow_m$, and if some

expression e is compiled to $C(e)$ then we must have “a commutative diagram”:

$$\begin{array}{ccc} e & \xrightarrow{*}_s & n \\ \downarrow & & \approx \\ C(e) & \xrightarrow{*}_m & n' \end{array}$$

where $n \approx n'$ states that values n and n' coincide. In our case, stating that the source and target values coincide means that if an expression e evaluates to an integer n then the machine execution halts with n on top of the final stack.

Exercise 9. Prove the following Lemma that relates the many-steps evaluation of arithmetic expressions with the many-steps evaluation of compiling such expression to the machine, under an arbitrary stack:

Lemma `compile_correct_gen` : $\forall (e: \text{expr}) (n: \mathbb{Z}) (s: \text{stack}),$ *Rocq*
`red_expr_star e n` $\rightarrow$
`red_asm_star stack (compile e) = ORes (n :: stack).`

Proceed by structural induction on the expression e .

This Lemma should be read as follows: if an expression e evaluates down to an integer n , then it must be the case the evaluation of the compilation of e , under an arbitrary stack s , yields a new stack $(n :: s)$. $\square$

Informally, we can sketch the proof of the previous Lemma as follows:

- case $e = n$:

We have `compile n` = `[APush n]`, hence the final stack is $n :: s$ which finishes the case.

- case $e_1 \circ e_2$:

We have `red_expr_star e1 n1` and `red_expr_star e2 n2`.

Thus we can invoke the induction hypothesis on e_1 and e_2 .

	stack before		stack after
<code>compile e₁</code>	s	by induction hypothesis	$n_1 :: s$
<code>compile e₂</code>	$n_1 :: s$	by induction hypothesis	$n_2 :: n_1 :: s$
<code>compile $\circ$</code>	$n_2 :: n_1 :: s$		$(\text{eval_op } n_1 \circ n_2) :: s$

6.1 Final Compiler Correctness

And here it is, our main result:

Exercise 10. Prove the following Lemma:

Lemma `compile_correct` : $\forall (e: \text{expr}) (n: \mathbb{Z}),$ *Rocq*
 `red_expr_star e n` $\rightarrow$
 `red_asm_star [] (compile e)` = `ORes [n]`.

The main difference with the previous Lemma is that evaluating the compilation of an expression `e` under an empty machine, *i.e.*, with an empty stack, shall produce a new stack containing only an integer `n`. Such an integer value is the arithmetic evaluation of `e`. $\square$